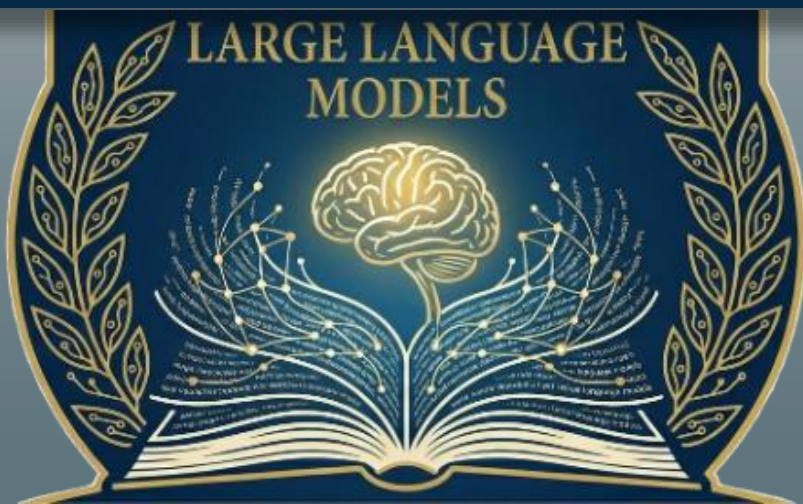


Reasoning



Mohammad Javad Dousti

Demystifying Chains, Trees, and Graphs of Thoughts

❑ Chain of Thought

- Understanding the standard prompting bottleneck and how *linear sequential reasoning* enables models to solve complex logical deduction tasks.

❑ Tree of Thoughts

- *Breaking linearity* through heuristic search algorithms, allowing models to explore multiple paths, look ahead, and backtrack from errors.

❑ Graph of Thoughts

- Moving toward synergistic reasoning by merging disparate thoughts, implementing *feedback loops*, and overcoming rigid topological boundaries.

Source: M. Besta et al. "Demystifying chains, trees, and graphs of thoughts." *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).

The Challenge of LLM Reasoning

Why standard prompting fails at complex logic?

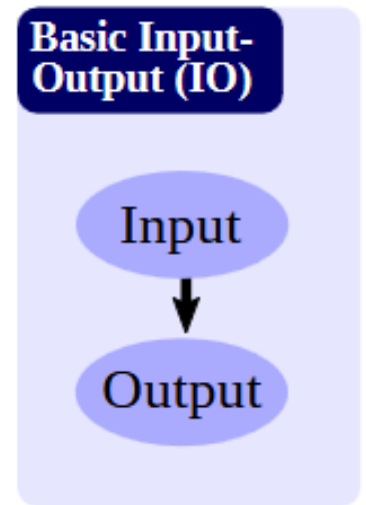
The Standard Prompting Bottleneck

❑ Left-to-Right Decoding

- Generative Transformer models predict the next token based entirely on the preceding context.
- In **standard prompting**, the model is forced to output a final answer immediately after the question.
- This “one-token-at-a-time” constraint means the model cannot allocate more computational power to harder problems before committing to a definitive output.

❑ Flat Scaling Curves

- For complex logic, multi-step mathematics, and advanced symbolic reasoning, simply increasing the parameter count of the model (e.g., from 10B to 100B) yields diminishing returns.
- Scaling alone cannot overcome the architectural limitation of requiring a direct map from question to answer without a working memory space.



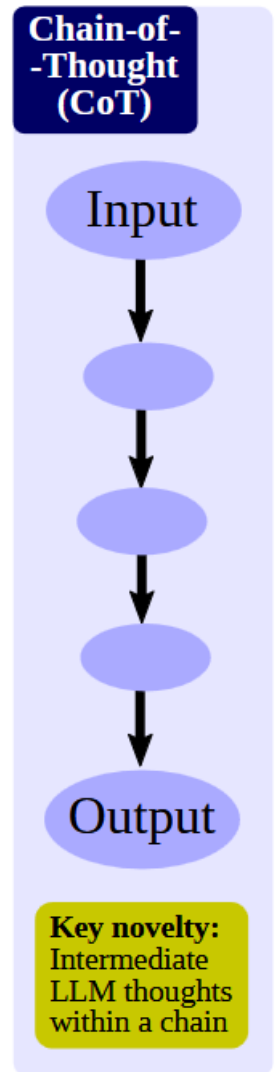
The Compute Limit

- *Implicit hypothesis behind reasoning research*

Standard prompting fundamentally forces an LLM to solve a complex math problem with $O(1)$ forward passes, regardless of the problem's actual computational complexity.

Enter Chain-of-Thought (CoT)

- ❑ Wei et al. (2022) proposed a simple yet profound shift:
 - Guiding the model to generate a series of **intermediate reasoning steps** before providing the final answer.
 - This allows the LLM to utilize its own output window as a sequential scratchpad, effectively converting $O(1)$ reasoning into $O(N)$ reasoning, where N is the length of the generated chain.
- ❑ The model now has the “*time*” (token space) to compute intermediate variables.



Source: J. Wei et al. "Chain-of-thought prompting elicits reasoning in large language models." *Advances in neural information processing systems* 35 (2022): 24824-24837.

Concrete Example: Math Word Problem

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

(Model often memorizes patterns but fails when numbers or logic are modified)

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

(Model calculates sequentially)

Implementing the Chain

❑ Few-Shot CoT

- The prompt includes a few hand-crafted exemplars. Each exemplar contains a question, followed by a step-by-step reasoning process, and finally the answer.
- This teaches the model the *format* and *style* of reasoning expected, heavily guiding its logic structure.

❑ Zero-Shot CoT

- Kojima et al. (2022) demonstrated that simply appending the trigger phrase “**Let's think step by step**” to the end of a query is often sufficient.
- This forces the model's internal probability distribution to favor generating explanatory text before concluding.

Source: T. Kojima et al. "Large language models are zero-shot reasoners." *Advances in neural information processing systems* 35 (2022): 22199-22213.

Zero-shot CoT Prompting

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Key Properties of CoT Prompting

❑ **Decomposability**

- Breaks down complex, multi-step problems into manageable intermediate steps, allocating more computation time specifically to harder queries.

❑ **Interpretability**

- Provides a transparent window into the model's behavior, allowing researchers to pinpoint exactly where a reasoning path went astray.

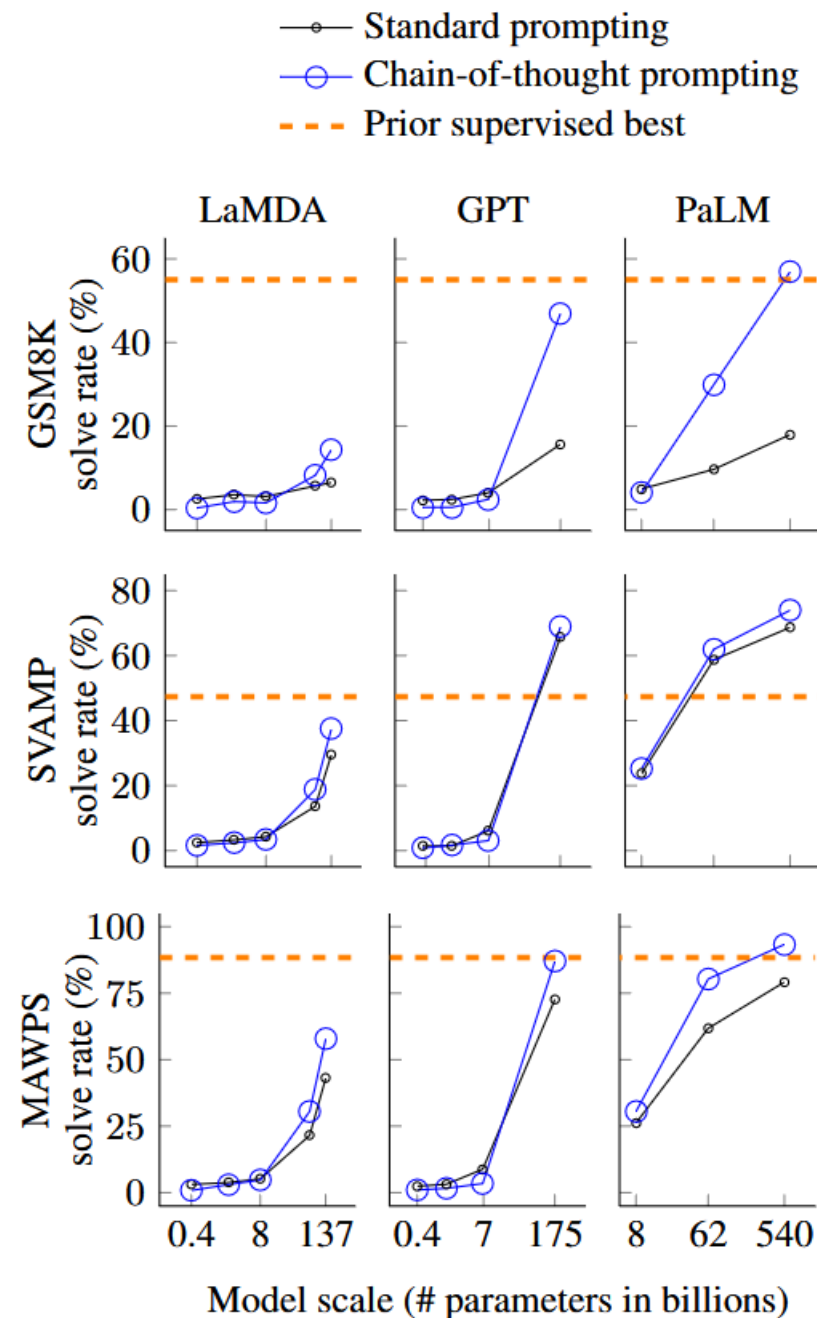
❑ **Generalizability**

- Applicable across diverse domains—including math word problems, commonsense reasoning, and symbolic manipulation—without fine-tuning.

The Emergence of Reasoning

□ An Emergent Property of Scale

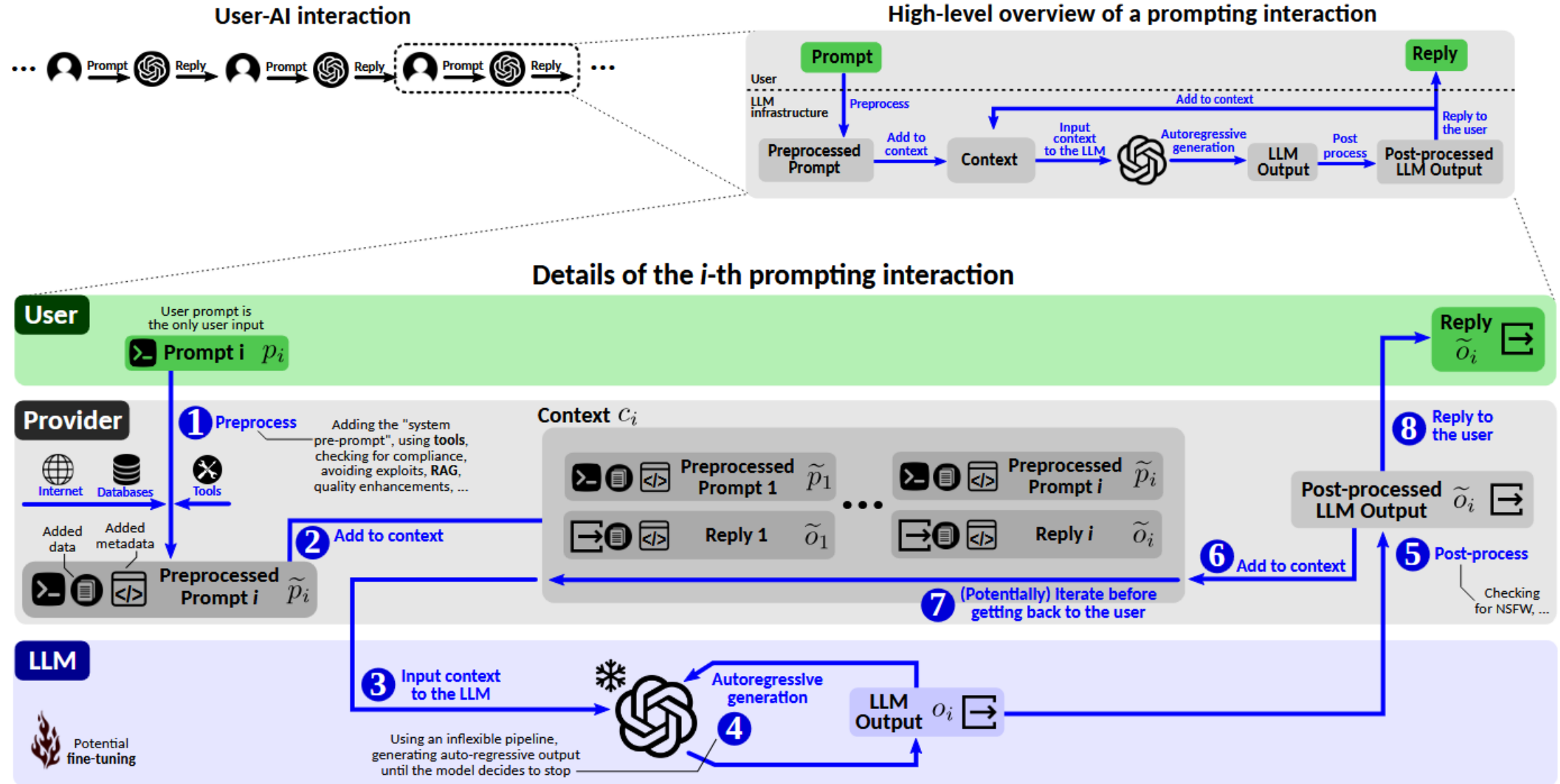
- Crucially, successful chain-of-thought prompting is an **emergent property**. The benefits do not scale linearly from small models.
- Below ~ 100 B parameters, models often produce fluent but illogical chains, frequently hallucinating or making basic arithmetic errors, which can actually degrade performance compared to standard prompting.



Ablation: Does the Reasoning Matter?

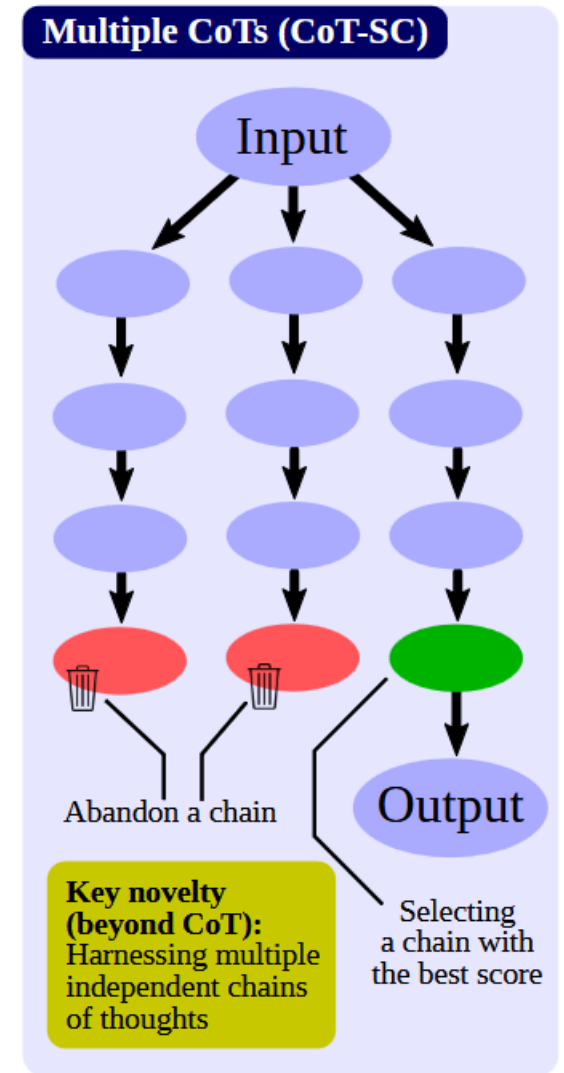
- Wei et al. conducted ablation studies to determine *why* CoT works. They tested providing correct equations but nonsensical natural language explanations in the prompt exemplars.
 - **Equations Matter:** Providing only the mathematical equations without natural language context still improved performance over standard prompting, but not as much as full CoT.
 - **Semantic Meaning Matters:** If the natural language explanation in the prompt is semantically incorrect or irrelevant (even if the final math is right), performance drops significantly.
 - **Conclusion:** The LLM is genuinely relying on the semantic structure of the generated text to route its subsequent token predictions. It is not just a formatting trick.

Overview of a general prompting pipeline



Bridging CoT to Trees: Self-Consistency

- ❑ Before moving to complex trees, Wang et al. (2022) introduced *Self-Consistency* as an extension to CoT.
- ❑ Instead of relying on a single greedy decoding path, the system samples multiple diverse reasoning paths (using a high temperature). It then selects the most consistent final answer by taking a **majority vote**.
- ❑ This acknowledges that there are multiple valid reasoning paths to the same correct conclusion, hinting at the need for non-linear exploration.



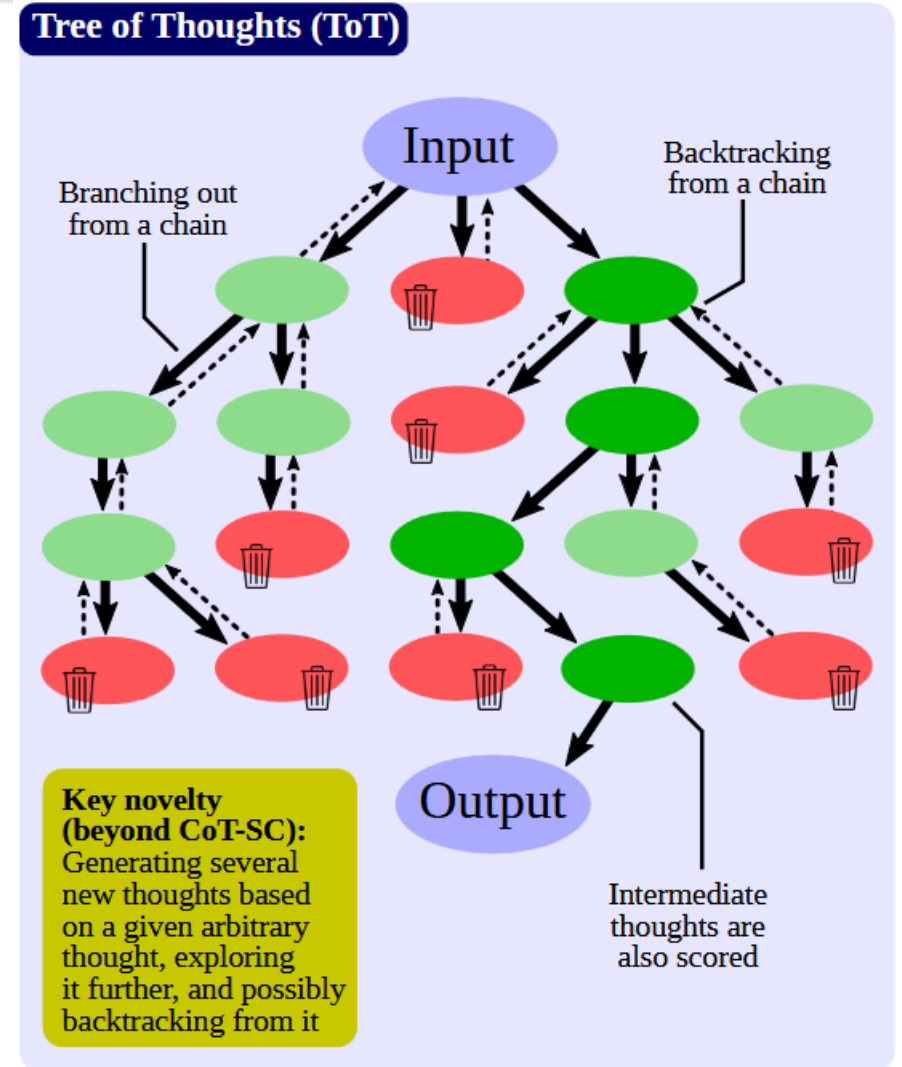
Source: X. Wang et al. "Self-consistency improves chain of thought reasoning in language models." *arXiv preprint arXiv:2203.11171* (2022).

Breaking Linearity: Tree of Thoughts

Moving from single left-to-right chains to multi-path exploration.

Reasoning as Tree Search

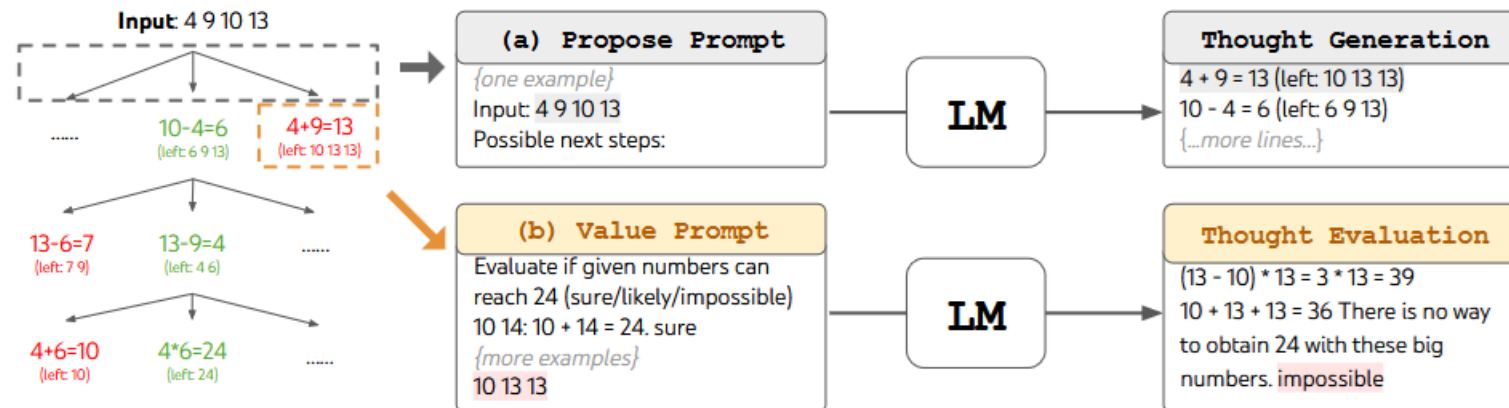
- ❑ Linear CoT suffers from a critical flaw:
 - If the model makes a mistake early on, the error propagates, and the model cannot backtrack.
- ❑ Yao et al. (2023) introduced the **Tree of Thoughts (ToT)**, framing reasoning as exploring a tree structure where intermediate states act as partial solutions.
- ❑ This allows LLMs to evaluate multiple reasoning paths simultaneously, look ahead, and deliberately backtrack from unpromising outcomes.



Source: Yao, Shunyu, et al. "Tree of thoughts: Deliberate problem solving with large language models." *Advances in neural information processing systems* 36 (2023): 11809-11822.

ToT Example: Game of 24

- ❑ **Goal:** Use four numbers and basic arithmetic (+, -, *, /) to reach exactly 24.
- ❑ **Input:** 4, 9, 10, 13
- ❑ Unlike standard math problems, Game of 24 requires extensive trial and error. CoT fails because it tries to guess the answer in one linear pass.
- ❑ ToT uses BFS to generate possible intermediate states (e.g., $10 - 4 = 6$. Remaining: 6, 9, 13).
- ❑ The *Evaluator* judges if reaching 24 is “impossible”, “maybe”, or “sure” from that subset.



Search Algorithms in ToT

❑ Breadth-First Search (BFS)

- Maintains a set of the most promising states at each depth level. Best for problems where the depth of the tree is bounded and relatively small.
- *Use case: Game of 24, where the tree depth is strictly limited to 3 steps (combining 4 numbers into 1).*

❑ Depth-First Search (DFS)

- Explores a single branch until a conclusion is reached or a heuristic evaluator determines the state is a dead-end, at which point it backtracks.
- *Use case: Creative writing or crosswords, where exploring a full line of thought is necessary before assessing viability.*

Core Components of ToT

❑ Thought Generators

- At every branch, the problem is decomposed. The system generates potential next states by either sampling various continuous thoughts via standard CoT, or by proactively proposing distinct, new strategic thoughts using a specialized prompt generator.

❑ State Evaluators

- Given different potential states, an evaluator acts as a heuristic. Instead of rigid formulas, the LLM itself is prompted to "reason" about the viability of each state (e.g., classifying a state as 'sure', 'maybe', or 'impossible'), guiding the search algorithm.

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	74%
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

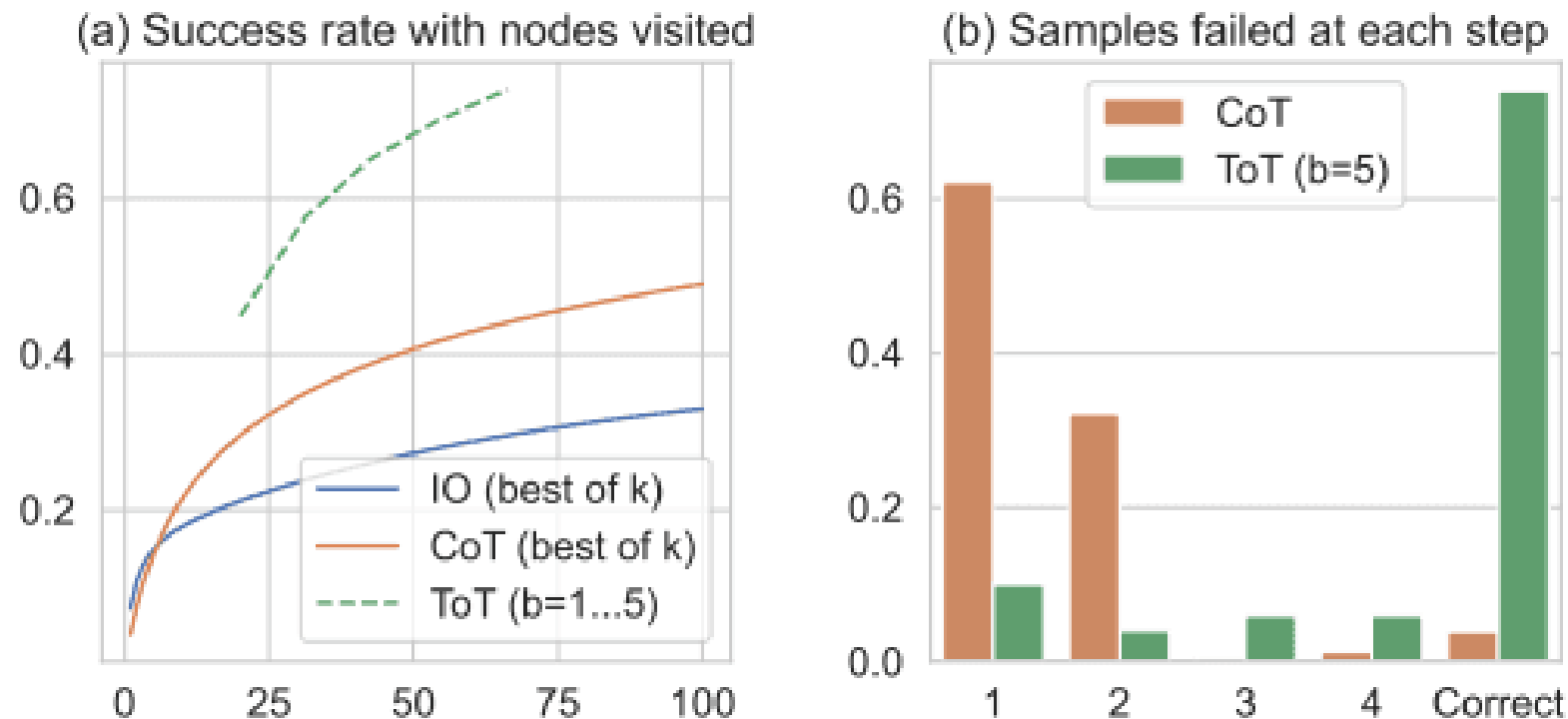


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

The Limitations of Tree Topologies

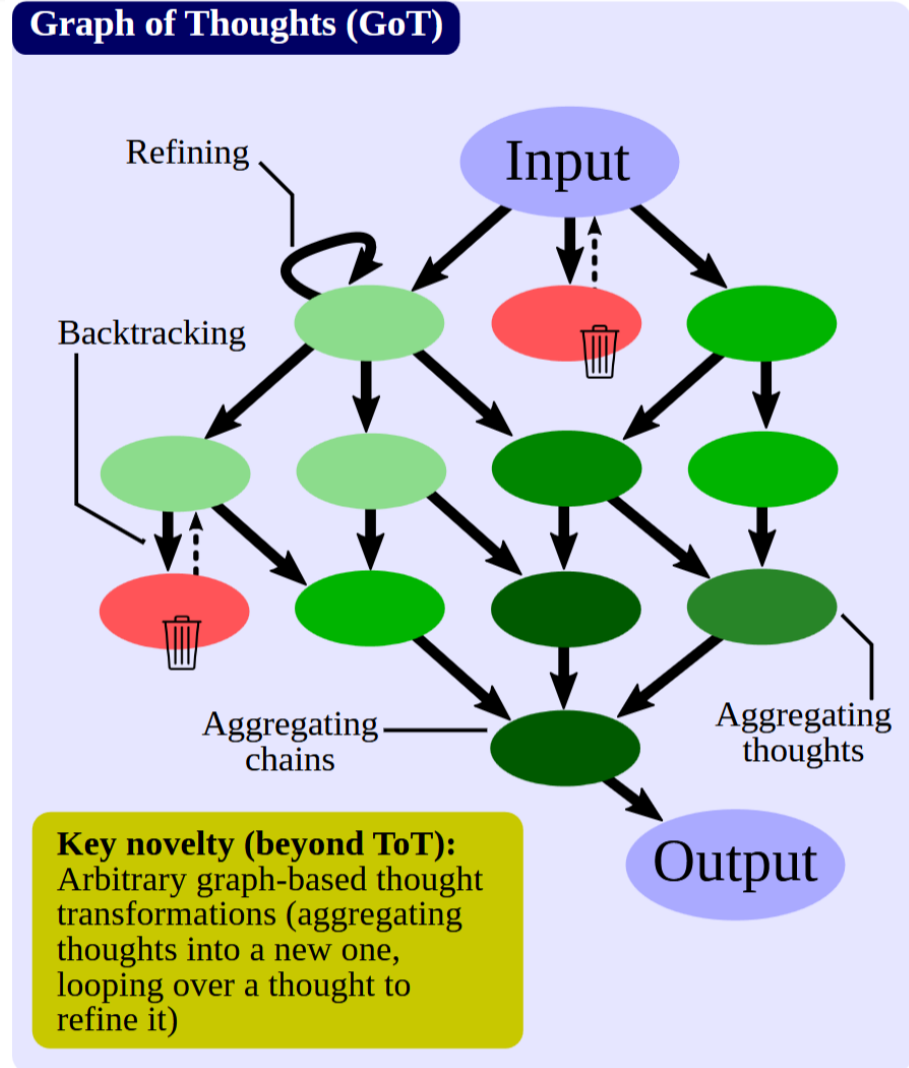
- ❑ **Siloed Information:** Trees impose a strict hierarchical structure, preventing the merging or synthesis of valuable information discovered in separate, parallel branches.
- ❑ **No Feedback Loops:** Tree structures cannot easily model feedback mechanisms or dynamically refine past thoughts based on subsequent, future discoveries.
- ❑ **High Inference Costs:** Without highly aggressive pruning strategies, exploring multiple branches leads to a combinatorial explosion of API calls and token generation costs.
- ❑ **Complex Synergies:** Many real-world multifaceted tasks require non-linear, synergistic combinations of partial solutions—a topology trees cannot support.

Networked Reasoning Graph of Thoughts

Synergizing information across arbitrary network topologies.

The Graph Abstraction

- ❑ Besta et al. (2023) introduce *Graph of Thoughts (GoT)*, enabling LLM thoughts to form arbitrary graph structures.
- ❑ By modeling units of information as vertices and dependencies as directed edges, GoT overcomes the rigid isolation of trees.
- ❑ This structure brings LLM reasoning closer to human brain mechanisms, which rely heavily on complex networks, recurrence, and synthesis of disparate ideas.



Source: M. Besta et al. "Graph of thoughts: Solving elaborate problems with large language models." *Proceedings of the AAAI conference on artificial intelligence*. Vol. 38. No. 16. 2024.

Advanced Thought Transformations

❑ **Aggregation**

- Constructing vertices with multiple incoming edges to combine the most promising traits from several distinct ongoing reasoning paths into a single, superior thought.

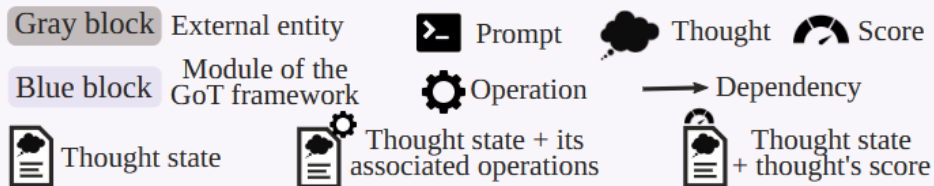
❑ **Refinement**

- Creating continuous feedback loops where thoughts are iteratively enhanced and corrected based on specific critique and evaluator feedback from the model.

❑ **Generation**

- Flexibly branching out new lines of reasoning from any optimal point in the entire graph history, not just strictly from the current deepest node.

Legend



API for Controller

- ➔ `//LLM params: model used, temperature, max tokens, api key, org, ...`
- ➔ `//LLM cost features: prompt token cost, response token cost, ...`
- ➔ `//Instances of Prompter + Parser + Graph of Operations,`
- ➔ `//Any additional input parameters (e.g., numbers to be sorted).`

Available operations when building the GoO (extensible)

- ➔ `Generate, Aggregate, Score, ...` //see Prompter API
- ➔ `KeepBest(N)` //preserves N best scoring thoughts
- ➔ `Repeat(k)` //Repeat a given operation k times, generating k thoughts.
 //For example, this enables "Aggregate" to generate multiple outcomes
 //of the combination operation. Each such thought is maintained
 //within the Graph Reasoning State and scored individually.

API for Prompter (extensible)

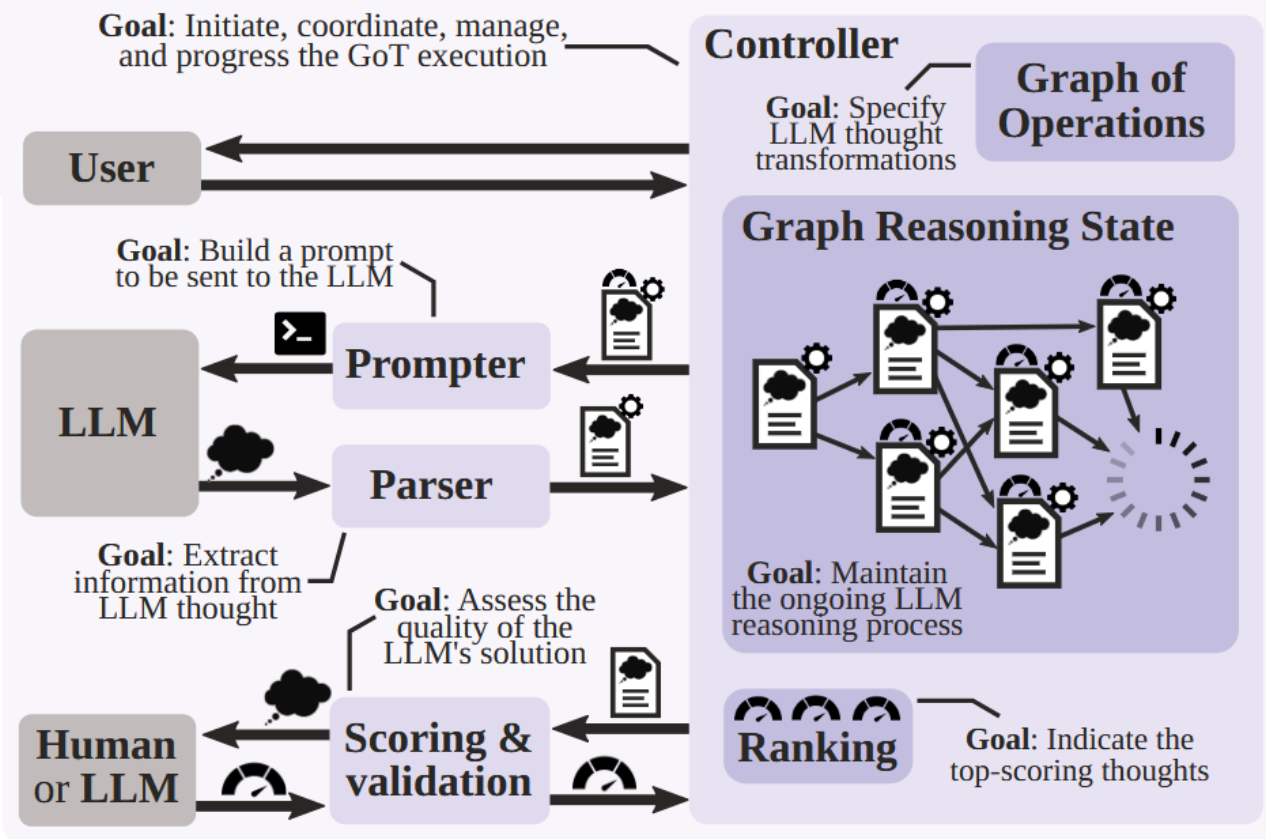
- ➔ `Generate(t, k)` //generate a prompt for k new thoughts, using thought t
- ➔ `ValidateAndImprove(t)` //generate a prompt to enhance thought t ,
- ➔ `Aggregate(t1, ..., tk)` //generate a prompt to combine thoughts $t1, ..., tk$
- ➔ `Score(t)` //score thought t
- ➔ `Validate(t)` //generate a prompt to validate the correctness of thought t

API for Parser (extensible)

`ParseGenerate, ParseImprove, ParseScore, ParseAggregate, ParseValidate, ...`

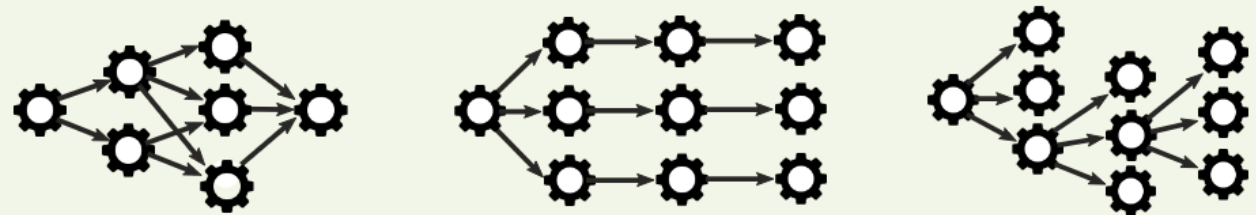
//Each of the above routines is responsible for parsing an LLM thought
 //to a corresponding Prompter routine (e.g., `ParseScore` parses `Score`).

Architecture overview



Specifying the structure of the Graph of Operations (GoO)

Graph of Operations enables seamless specification of not only GoT, but also existing schemes such as CoT, CoT-SC, ToT



GoT Example: Array Sorting

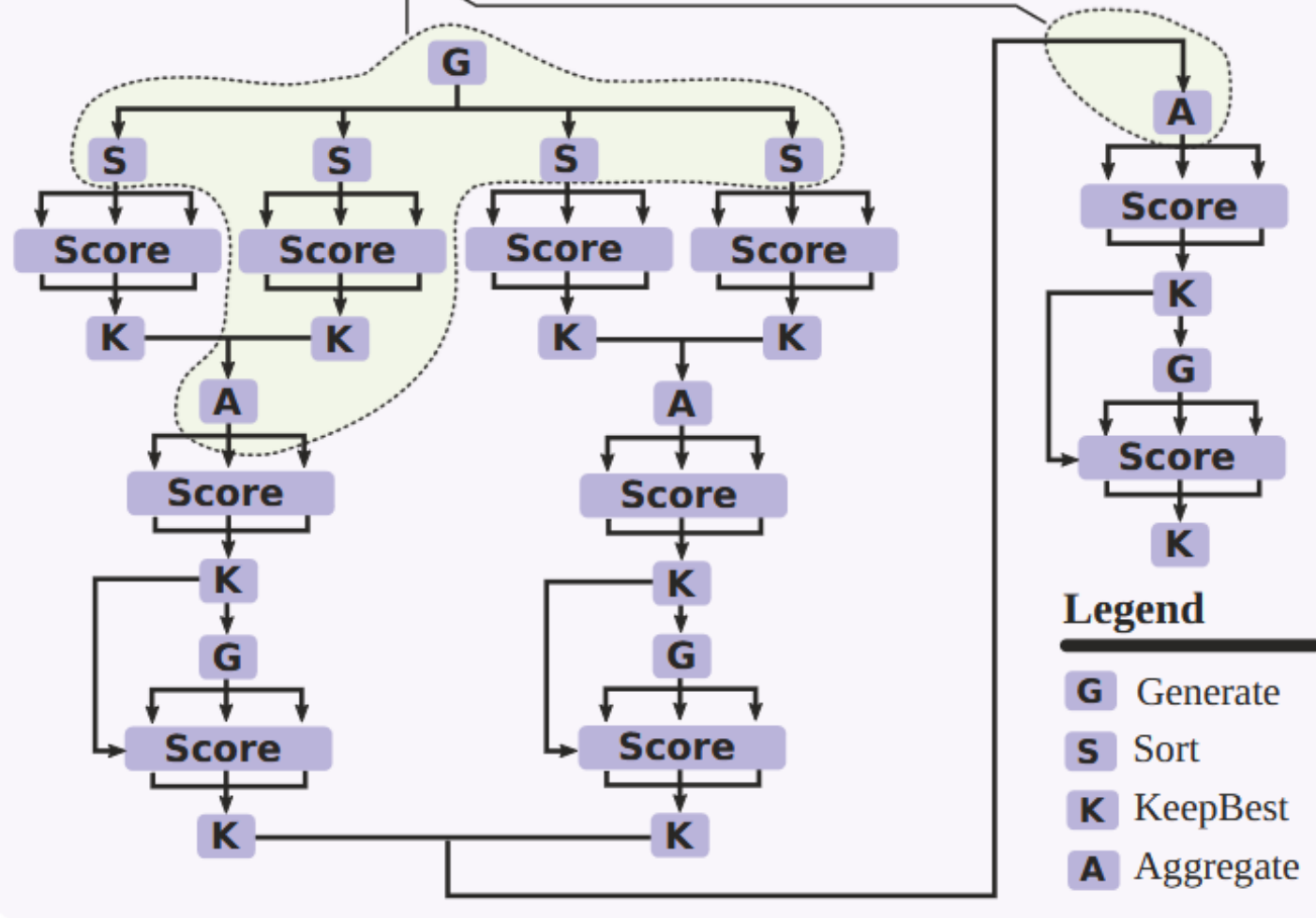
❑ Merge Sort Topology

- Sorting a large array of 64 elements is highly error-prone for an LLM in a single pass (CoT).
- GoT allows the topology to mirror a classic **Merge Sort algorithm**. The list is split into sub-arrays (branches), sorted individually, and then the critical operation happens: **Aggregation**.
- The LLM is prompted to merge two previously sorted sub-arrays into a single sorted array, effectively creating a graph structure with converging nodes.
- $O(N \log N)$: By mapping the prompting strategy to efficient computer science algorithms, GoT achieved a 62% improvement in sorting accuracy over ToT.

Graph of Operations (GoO) for sorting 64 numbers

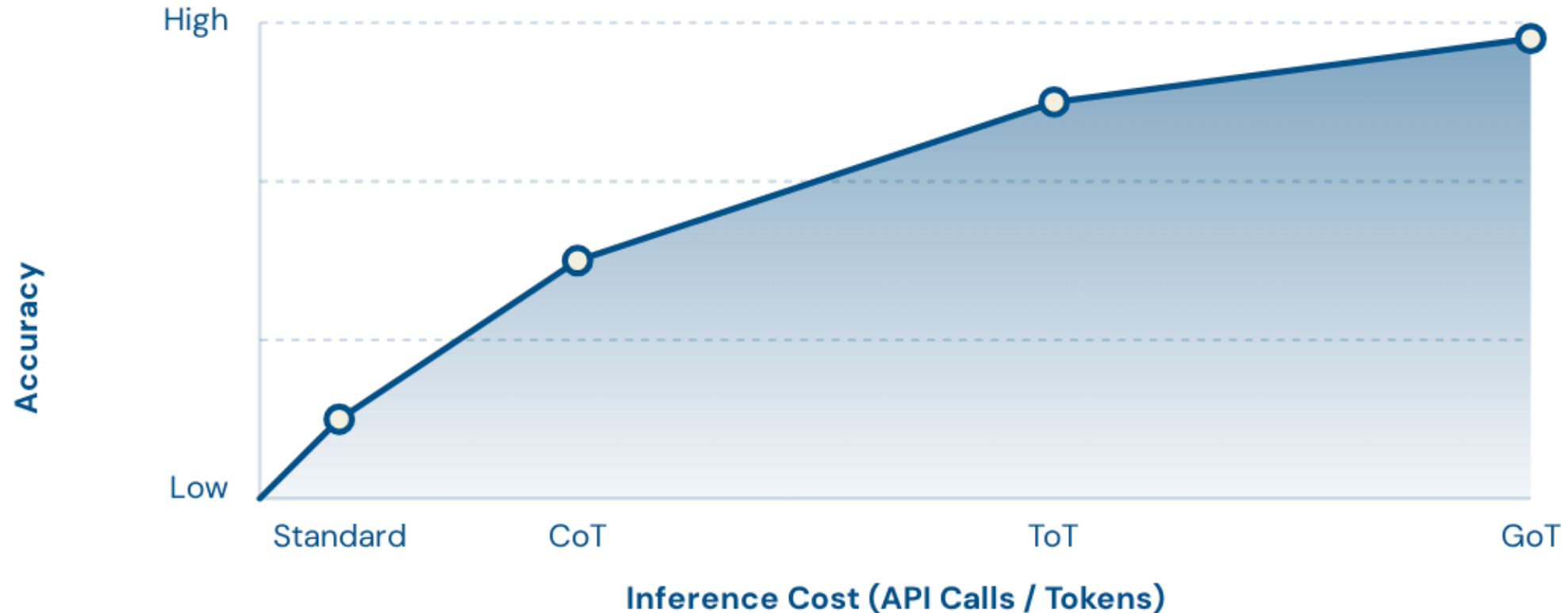
Details of the highlighted part of GoO are below

Note that this is an example graph decomposition. The structure of connections can be arbitrarily modified.



The Trade-off: Inference Cost vs. Accuracy

- Advanced topologies yield higher accuracy but require logarithmically increasing inference compute. Engineering applications must balance acceptable latency and cost against task complexity.



Taxonomy of Reasoning Schemes

Paradigm	Topology Structure	Primary Advantage	Key Limitation
Standard Prompting	Single Node (Direct)	Fast, low token cost, simple.	Fails entirely at complex, multi-step logic.
Chain of Thought (CoT)	Linear Path	Interpretable, enables reasoning at scale.	Errors propagate; no backtracking.
Tree of Thoughts (ToT)	Hierarchical Tree	Path exploration, lookahead, backtracking.	High inference costs; branches remain siloed.
Graph of Thoughts (GoT)	Arbitrary DAG / Network	Synergistic combination, feedback loops.	High implementation complexity & overhead.

Future Directions in Topology

❑ Internalizing Search

- Moving from explicit prompt-based search wrappers (System 2) back to implicitly internalizing these search traces during pre-training or fine-tuning (System 1).

❑ Algorithm of Thoughts (AoT)

- Generating the entire search process—including backtracking and path evaluations—within a single context window to drastically reduce API latency.

❑ Agentic Workflows

- Extending GoT topologies beyond internal thoughts to external API calls, code execution, and multi-agent debate structures.

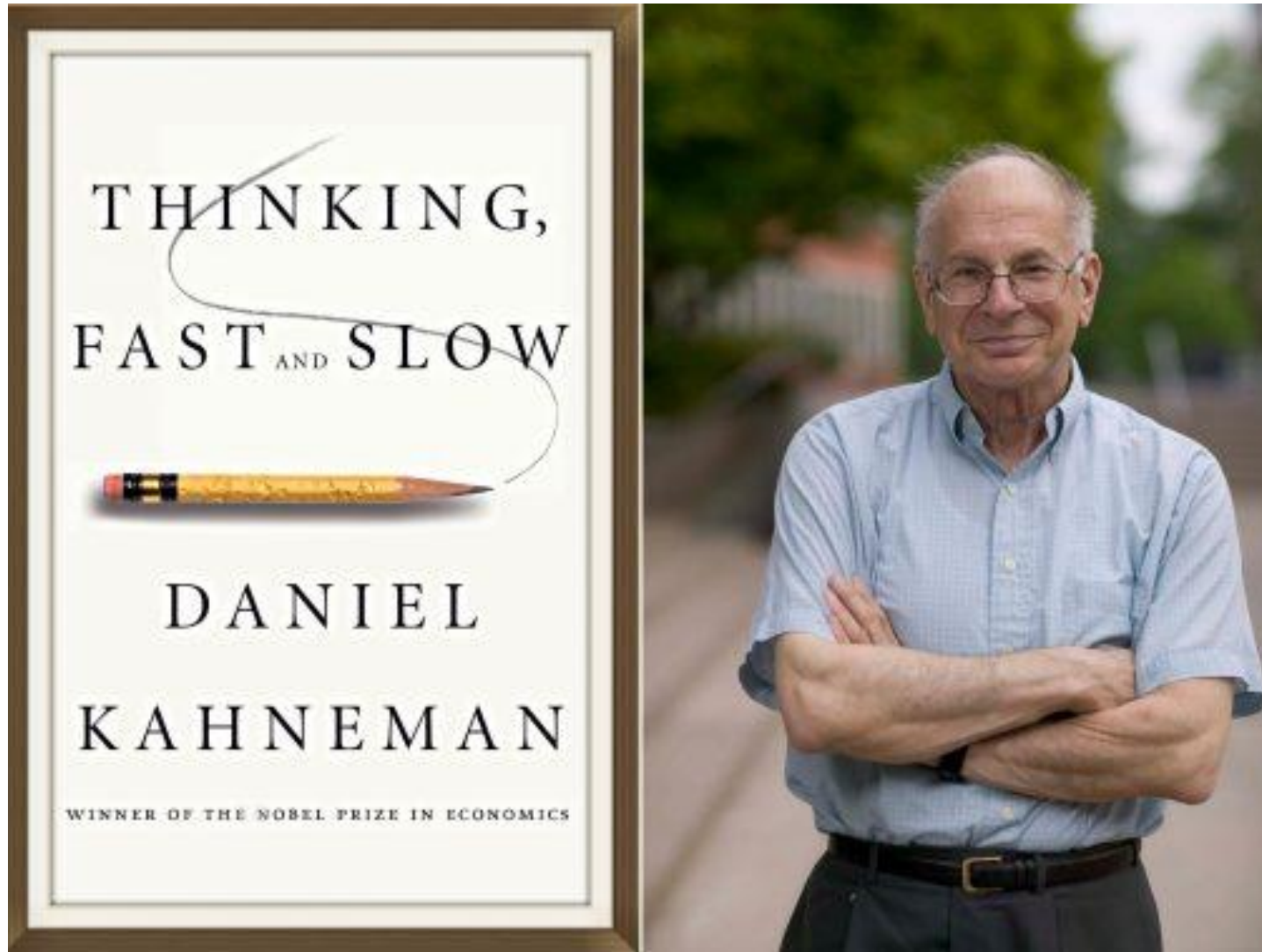
The Dawn of System 2 AI

Exploring Advanced Reasoning Architectures:
OpenAI o1/o3, DeepSeek R1, and the Math of GRPO

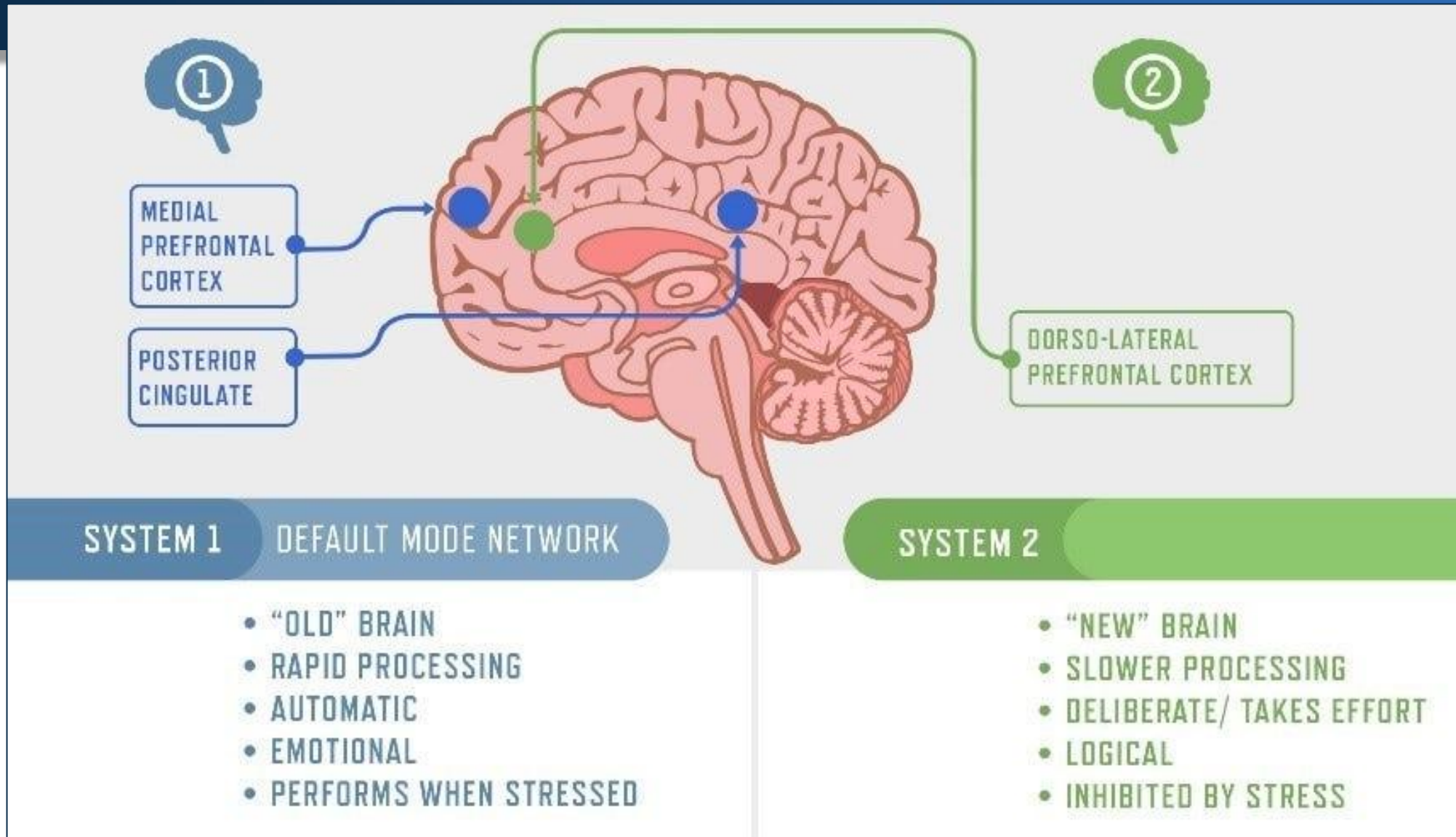
The Paradigm Shift

From static prediction to deliberate, verifiable search.

Thinking, Fast and Slow



Watch this on YouTube: [Thinking, Fast and Slow | Daniel Kahneman | Talks at Google \(2011\)](#)



Source: <https://www.linkedin.com/pulse/system-1-brain-2-brains-daniele-bacchion-6sqje>

System 1 vs. System 2 Thinking

System 1 (GPT-4o, Claude 3.5)

- Fast, automatic, intuitive.
- Standard LLMs generate answers in a single forward pass, relying purely on pattern recognition learned during pre-training. They "think" as they speak.
 - Low latency, high fluency
 - Struggles with complex logic
 - Fixed $O(1)$ compute per token

System 2 (o1, R1)

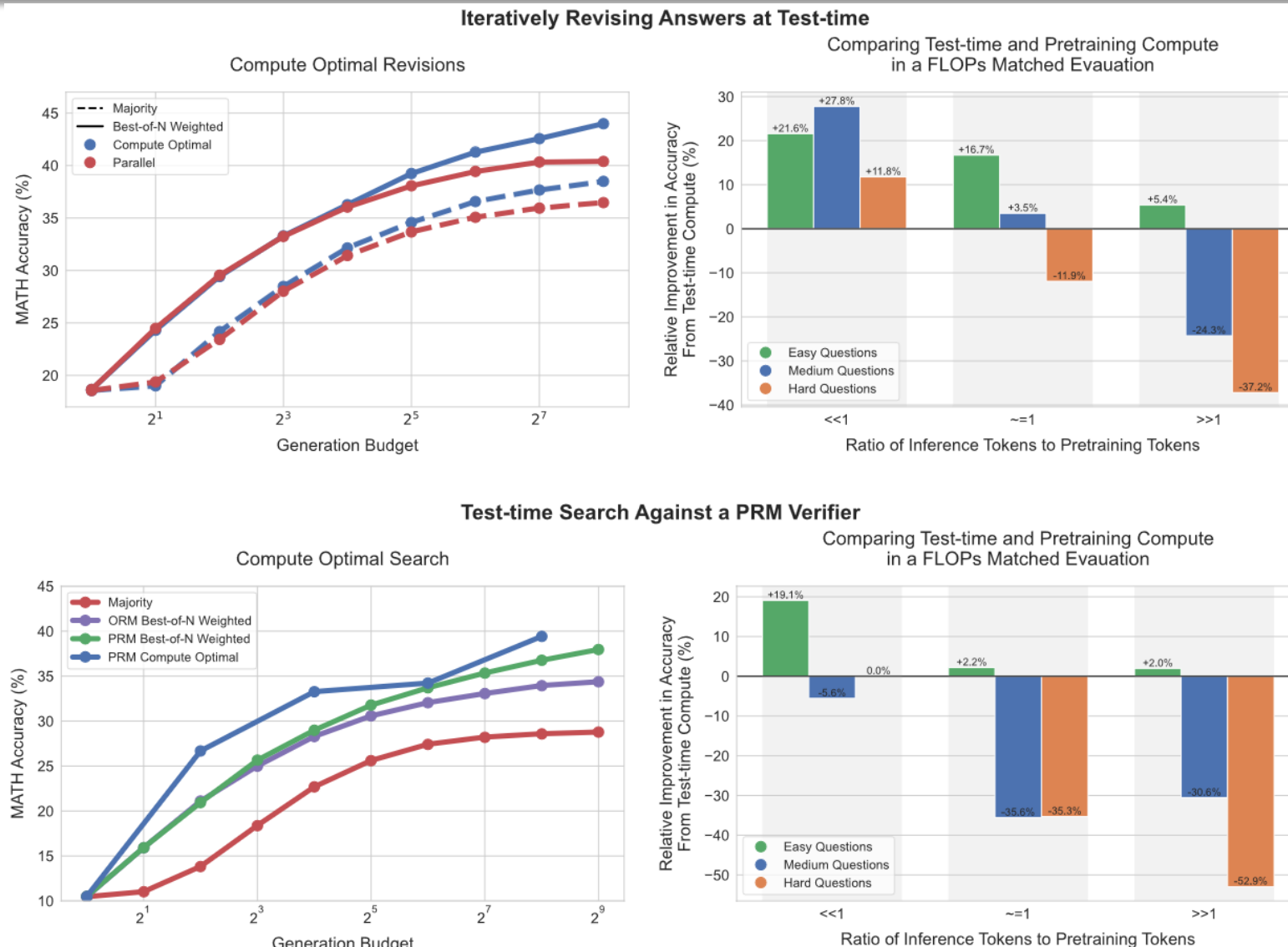
- Slow, deliberate, analytical.
- Reasoning models pause to "think" in a hidden scratchpad, explore paths, verify steps, and backtrack before committing to an output.
 - Higher latency (seconds to minutes)
 - Excels at math, coding, science
 - Scales with Test-Time Compute

The Inference Scaling Law

- ❑ **A New Dimension of Scaling**
- ❑ Historically, AI progress relied almost entirely on scaling **Training Compute** (larger datasets, bigger clusters).
- ❑ Recent research proves a second scaling law: **Test-Time Compute**.
- ❑ The longer a model is allowed to *think* (generate intermediate reasoning tokens) during inference, the higher its accuracy on complex benchmarks.
- ❑ This effectively shifts the compute bottleneck from the pre-training cluster to the inference server.

Source: Snell et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*.

The Inference Scaling Law (2)



Source: Snell et al. (2024). Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters.

Process Rewards & OpenAI o1

Teaching models to verify their own steps.

Outcome vs. Process Supervision (1)

❑ Outcome Reward Models (ORM)

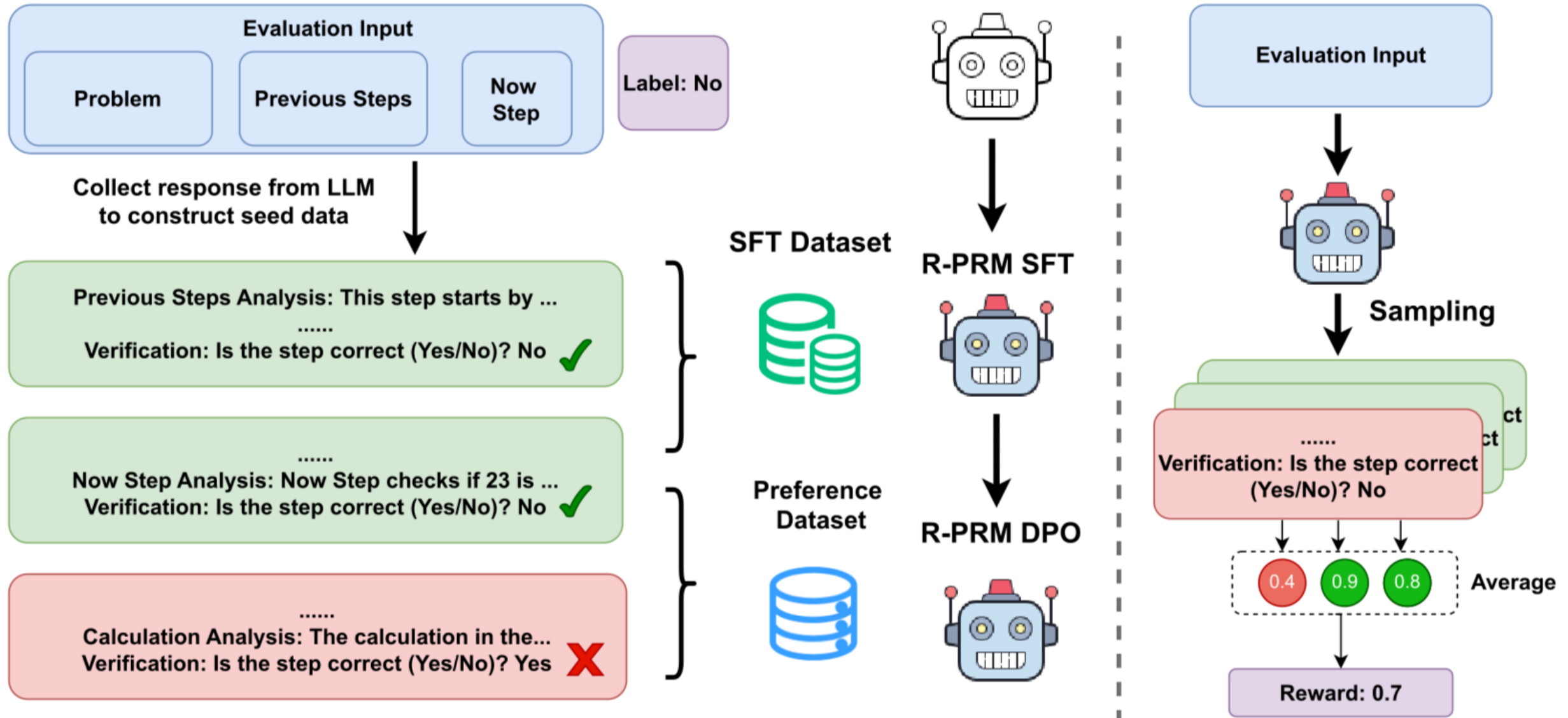
- Traditional RL optimizes for the final answer. The model is rewarded if it gets the right answer, penalized if wrong.
- This suffers from **reward hacking**—the model might guess the right answer using flawed logic.

❑ Process Reward Models (PRM)

- PRMs evaluate and reward *every single intermediate step* of the reasoning chain.
- The model learns to verify its own logic step-by-step, vastly reducing hallucinations in complex math and coding tasks.

Source: Lightman et al. (2023). *Let's Verify Step by Step*. OpenAI.

Outcome vs. Process Supervision (2)



The Value of Process

- ❑ “Process supervision significantly outperforms outcome supervision for training models to solve problems from the challenging MATH dataset... it provides more precise feedback and naturally mitigates alignment tax.”

Source: Lightman et al. (2023). *Let's Verify Step by Step*. OpenAI.

The OpenAI o1 Breakthrough

❑ Beyond Static Prompting

- Prior techniques like Zero-Shot CoT ("Think step by step") forced the model to generate text, but the model wasn't intrinsically trained to reason over long horizons.
- OpenAI o1 (formerly Q* / Strawberry) was trained end-to-end using *Reinforcement Learning (RL)* specifically to execute a private chain of thought before responding.
- It learns to recognize its own mistakes, try alternative strategies, and break down complex queries autonomously.

❑ The Hidden CoT

- o1's reasoning tokens are hidden from the user to maintain competitive advantage and prevent raw distillation of its exact thought process.

The o1 Performance Leap

- ❑ On the highly rigorous American Invitational Mathematics Examination (AIME), GPT-4o solved only 13% of the problems.
- ❑ OpenAI o1 solved 83%.
 - It also achieved 89th percentile in Codeforces competitive programming.
- ❑ This marked the first time an AI performed at a PhD level on complex physics, chemistry, and biology benchmarks (GPQA Diamond).

The Evolution: OpenAI o3

o3 Architecture Updates

- Released in early 2025, o3 refined the RL reasoning process, significantly boosting performance on SWE-bench (software engineering) to 71.7% (up from o1's 48.9%).
- It handles STEM reasoning with far greater reliability, practically eliminating logic loops where the model gets stuck repeating the same failed derivation.

The "Reasoning Effort" Dial

- o3-mini introduced explicit developer control over Test-Time Compute via a parameter: **Low, Medium, High**.
- Moving from low to high effort raises accuracy on benchmarks by 10-30%, allowing developers to explicitly trade latency and API cost for intelligence.

DeepSeek V3 & R1

The Open-Source Reasoning Revolution.

The Foundation: DeepSeek-V3

❑ Architectural Efficiency

- Before R1, DeepSeek released V3, establishing a highly efficient base.
- **Mixture of Experts (MoE):** 671B total parameters, but only activates ~ 37 B per token. It uses an auxiliary-loss-free strategy for dynamic load balancing.
- **Multi-head Latent Attention (MLA):** A sparse attention mechanism that drastically reduces KV-cache memory overhead for long-context inference.

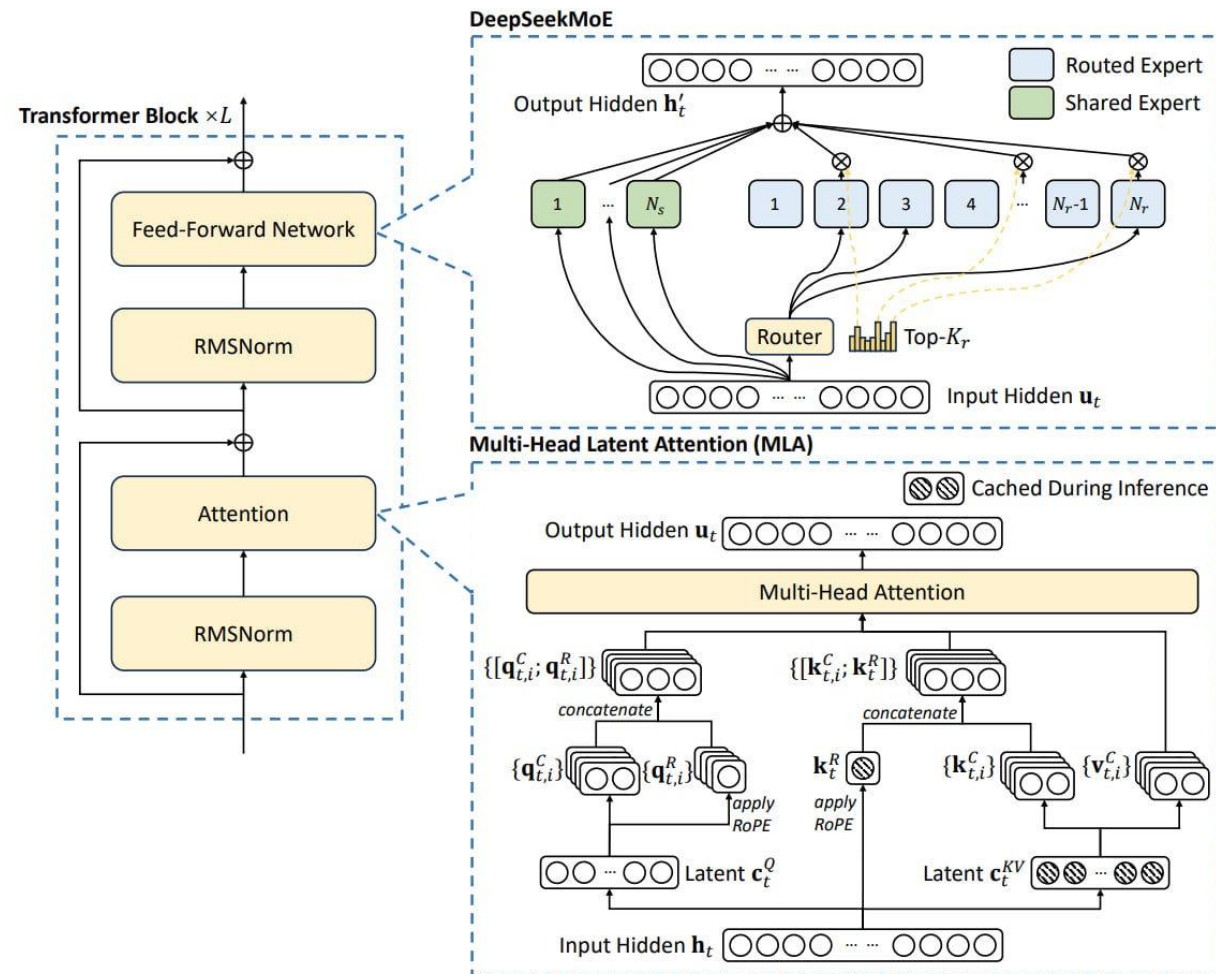


Figure 2 | Illustration of the basic architecture of DeepSeek-V3. Following DeepSeek-V2, we adopt MLA and DeepSeekMoE for efficient inference and economical training.

The R1-Zero Breakthrough

❑ Reasoning without Supervised Data

- The AI community assumed advanced reasoning required massive datasets of human-annotated chain-of-thought examples (SFT).
- DeepSeek R1-Zero proved this false. They applied *pure Reinforcement Learning* directly to the V3 base model without any initial supervised fine-tuning.
- Using a simple rule-based reward system ("Is the final math answer correct?"), the model autonomously developed complex reasoning behaviors.

Source: DeepSeek-AI et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.

Emergent Behavior

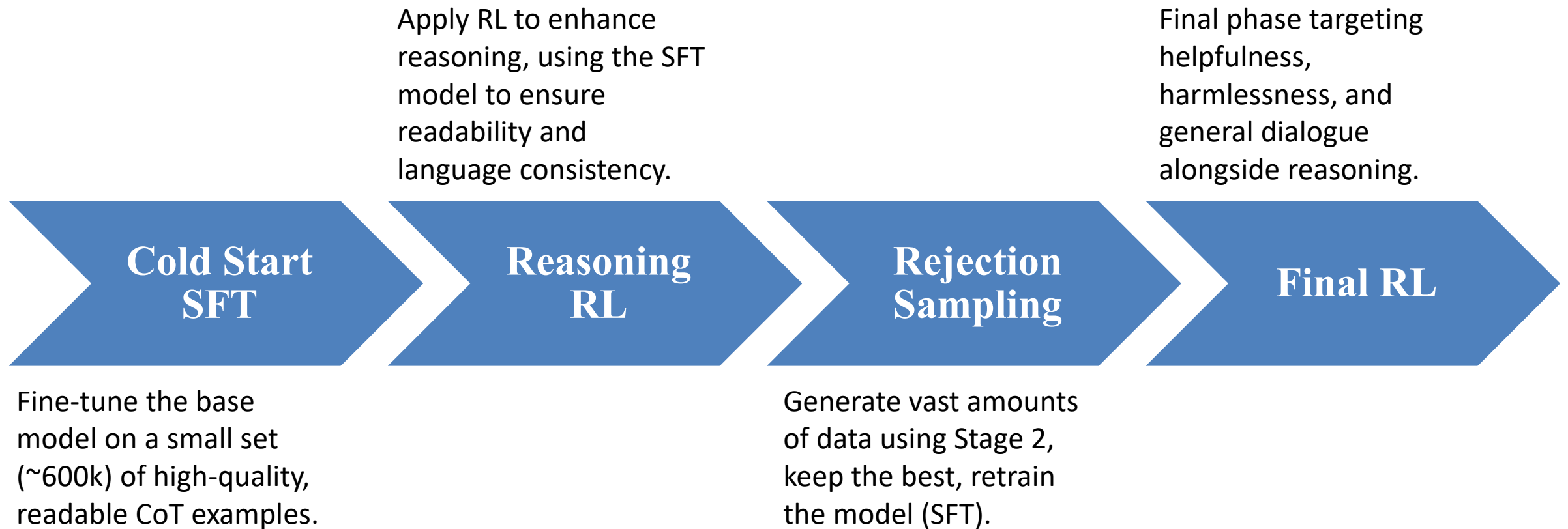
- ❑ “R1-Zero naturally evolved the tag, learned to allocate more tokens to difficult problems, and spontaneously demonstrated self-reflection and error correction mid-generation.”
- ❑ — DeepSeek R1 Technical Report (The "Aha Moment")

Limitations of Pure RL (R1-Zero)

- ❑ While R1-Zero proved reasoning emerges via pure RL, the resulting outputs were practically unusable for standard consumer applications:
 - **Language Mixing:** The model would frequently swap between English, Chinese, and code syntax mid-sentence, prioritizing internal logic efficiency over human readability.
 - **Poor Readability:** Outputs were disjointed, lacking consistent formatting or clear structural flow.
 - **Alignment Issues:** Without supervised tuning, the model struggled to maintain a consistent persona or adhere to safety guidelines outside of strict mathematical accuracy.

The DeepSeek R1 Pipeline

- ❑ To fix R1-Zero, DeepSeek developed a 4-stage pipeline for the final R1 model.



Demystifying GRPO

The Memory Wall of PPO

Traditional PPO

- Proximal Policy Optimization (PPO) typically requires a separate “Critic” (Value) model, identical in size to the main model, to estimate the baseline value of states.
- For a massive 671B parameter model, running a concurrent 671B Critic model in memory is computationally unfeasible and incredibly expensive.

DeepSeek's GRPO

- **Group Relative Policy Optimization (GRPO)** eliminates the Critic model entirely.
- It computes the baseline implicitly. It samples a group of outputs for the same prompt, scores them, and normalizes the scores relative to the group average.

GRPO vs. DPO (1)

- ❑ While DPO (Direct Preference Optimization) is excellent for aligning models to human conversational tones, it falls short when trying to teach a model advanced logical reasoning. DeepSeek developed GRPO to overcome three critical limitations:
 - **The Data Bottleneck (Rule-Based vs. Human Preference):** DPO requires static, offline datasets of paired responses (e.g., a human deciding Answer A is better than Answer B).
 - Generating accurate preference pairs for complex, 50-step mathematical proofs or software engineering tasks is incredibly slow and expensive. GRPO, on the other hand, relies on **rule-based rewards** (e.g., "Did the code pass the unit test?" or "Is the final extracted number 42?").
 - This allowed DeepSeek to train the model on millions of math problems without needing a human to rate the steps.

GRPO vs. DPO (2)

- **Active Exploration vs. Static Mimicry:** DPO only teaches the model to mimic the "winning" answers already present in its dataset. It doesn't encourage the model to invent new ways of thinking. GRPO is an active Reinforcement Learning algorithm.

The GRPO Advantage Formula

- For a given question q , GRPO samples a group of G outputs: $y_1, \dots, y_G$.
- It evaluates each output using a (rule-based) reward function to get scores $r_1, \dots, r_G$.
- The advantage $\hat{A}_i$ for each output is calculated by standardizing these scores:
$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$
- By using the group mean as the baseline, GRPO completely removes the need for a separate parameterized value model.

Rule-Based Reward Signals in GRPO

❑ Accuracy Rewards

- The primary driver. For math problems with deterministic answers, the reward is simply 1 if the final extracted number matches the ground truth, and 0 otherwise.

❑ Format Rewards

- Enforces structure. The model is rewarded for correctly encapsulating its thought process within `<think>...</think>` tags, ensuring the chain remains distinct from the final answer.

❑ Language Penalties

- To combat the language mixing seen in R1-Zero, a slight negative reward is applied if the proportion of the target language (e.g., English) in the thought process falls below a threshold.

Distillation & Ecosystem Impact

Bringing System 2 to local hardware.

Democratizing System 2

❑ The Power of Distillation

- Transferring the reasoning traces of massive 671B parameter models into compact, dense architectures that can run efficiently on consumer hardware.

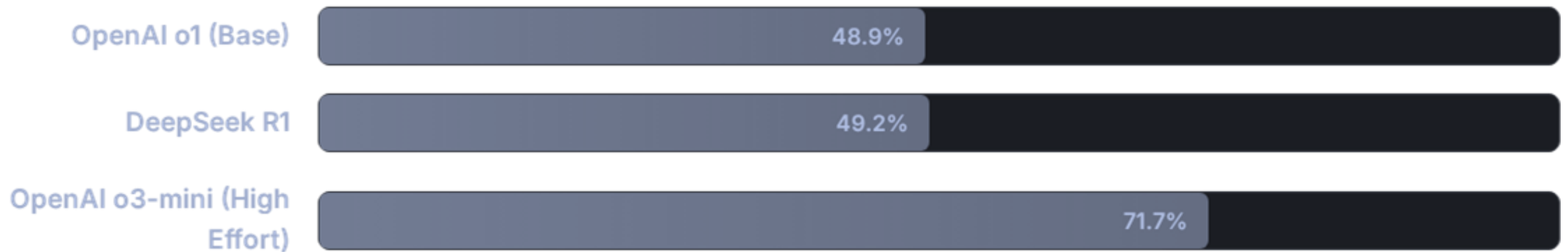
❑ Distilling R1 into Llama/Qwen

- DeepSeek used R1 to generate high-quality reasoning traces and used them to directly fine-tune open-source models like Llama-3 and Qwen.
- The resulting distilled models (1.5B to 70B) inherited R1's reasoning patterns without needing to undergo the expensive RL phase themselves.
- This allows startups and researchers to run state-of-the-art reasoning locally on single-node hardware.

❑ **1.5B - 8B Models:** Runs efficiently on edge devices and MacBooks.

❑ **32B - 70B Models:** Rivals GPT-4o logic capabilities natively.

SWE-bench Verified (Software Engineering)



- While DeepSeek R1 matched the initial o1 release using vastly less training compute, OpenAI's iterative o3 architecture significantly pushed the boundary forward on coding and agentic tasks.

The Future of Reasoning

The Next Frontier

❑ **Agentic Workflows**

- Integrating reasoning loops directly with tool-use. The model pauses to write code, runs it, reads the error, and adjusts its thought process automatically.

❑ **Thought Compression**

- Reasoning uses massive amounts of output tokens. Future research aims to internalize these search traces back into the base weights to reduce inference latency.

❑ **Verifier Scaling**

- As generation becomes cheaper, scaling the *Verifier* (the model that scores the outputs) becomes the primary focus for achieving superhuman accuracy in formal logic.

Taxonomy of Modern Models

Model Class	Architecture/Approach	Key Breakthrough	Primary Use Case
GPT-4o / Claude 3.5	Dense / MoE, Next-Token SFT	Multimodal speed, instruction following.	General chat, rapid drafting.
OpenAI o1	RL-trained Private CoT	Test-time compute scaling laws.	Complex math, PhD-level science.
DeepSeek R1	MoE, GRPO Reinforcement Learning	Reasoning emergence without SFT data.	Open-source logic, cost-efficiency.
OpenAI o3-mini	Optimized RL, Process Rewards	Adjustable reasoning effort, Agentic tasks.	Software engineering, SWE-bench.